Full Stack Developer in 30 Days

React (frontend) + Spring Boot (backend) + PostgreSQL — one working app deployed live.
Each day: tasks on both sides of the stack + a short YouTube video (≤ 25 min).

30 Days Full roadmap | **4 Weeks** Clear phases | **2 Stacks** React + Spring | **Auth + CRUD** Core features | **Deployed** Live app

Full Stack Windows Setup

Install everything you need on day 1

1 — Install Node.js LTS
nodejs.org → Windows LTS installer → adds npm automatically
nodejs.org

2 — Install JDK 21 LTS
adoptium.net → Windows x64 MSI → JDK installed + JAVA_HOME set
adoptium.net

3 — Install Cursor IDE
cursor.com/download → Cursor for React + AI assistance
cursor.com

4 — Install IntelliJ IDEA
jetbrains.com/idea → Community Edition → for Spring Boot
jetbrains.com

5 — Install PostgreSQL
postgresql.org → Windows installer → password: postgres, port 5432
postgresql.org

6 — Install Postman
postman.com → test backend APIs before connecting to React
postman.com

7 — Create React App
npx create-react-app frontend in one terminal tab

8 — Generate Spring App
start.spring.io → Maven, Java 21, Web + JPA + PostgreSQL → open in IntelliJ
start.spring.io

Key tools: Node.js · React · Spring · PostgreSQL · Axios · Tailwind · Railway · Vercel ·

WEEK 1 —React Frontend Fast Track

Days 1–7

Day 1 — Setup

Dev Environment Setup

- Install Node.js LTS + Cursor IDE + JDK 21
- Create React app: npx create-react-app my-app
- Run both React (port 3000) and Spring (8080)

Traversy · Full Stack Intro · ~20min

Day 2 — React

React Components & State

- Functional components, props, useState hook
- Build: Login form with controlled inputs
- Handle submit and log form data to console

Web Dev Simplified · React State · ~15min

Day 3 — React

React Router & Pages

- npm install react-router-dom
- Routes: / Home, /login Login, /dashboard
- Protected route: redirect if not logged in

Web Dev Simplified · Router · ~15min

Day 4 — Style

Tailwind CSS Styling

- Configure Tailwind in React app
- Style the login form and navigation
- Responsive layout: mobile and desktop

Fireship · Tailwind in 100 sec · ~2min

Day 5 — Spring

Spring Boot Backend Setup

- Generate project at start.spring.io: Web, JPA, PostgreSQL
- Create User entity: id, name, email, password
- JpaRepository + REST controller skeleton

Amigoscode · Spring Boot Intro · ~20min

Day 6 — Connect

CORS & First API Call

- Add @CrossOrigin to Spring controller
- React: fetch POST /api/users from form submit
- Display API response in the React UI

Traversy · CORS Fix · ~10min

Day 7 — Project

Week 1 Project: Register Flow

- React register form → POST to Spring API
- Spring saves user to PostgreSQL DB
- Show success message from API response

Amigoscode · Full Stack Demo · ~20min

WEEK 2 —JWT Auth + Full CRUD Stack

Days 8–14

Day 8 — Auth

JWT Auth Backend

- Add Spring Security + jjwt dependency
- POST /auth/login → validate → return JWT
- Protect endpoints with @PreAuthorize

Amigoscode · JWT Spring · ~22min

Day 9 — Auth

JWT Auth Frontend

- Store JWT in localStorage after login
- Attach token to all API calls: Authorization header
- Redirect to /login if token missing or expired

Web Dev Simplified · JWT React · ~20min

Day 10 — React

Axios for API Calls

- npm install axios — better than raw fetch
- Create api.js: axios instance with baseURL
- Interceptor: auto-attach JWT to every request

Traversy · Axios Crash · ~15min

Day 11 — Spring

CRUD Backend: Tasks API

- Task entity: id, title, done, userId (FK)
- Endpoints: GET, POST, PUT, DELETE /api/tasks
- Only return tasks belonging to logged-in user

JavaGuides · CRUD API · ~20min

Day 12 — React

CRUD Frontend: Task List

- Fetch and display tasks on Dashboard page
- Add task form: POST, update done: PUT
- Delete button: DELETE, remove from UI state

Traversy · React CRUD · ~20min

Day 13 — React

React Context for Auth State

- AuthContext: user, token, login, logout
- Wrap App in AuthProvider
- useContext in Navbar to show/hide links

Web Dev Simplified · Context · ~14min

Day 14 — Project

Week 2 Project: Task Manager App

- Working auth: register, login, logout
- Protected task dashboard — full CRUD
- Tasks tied to logged-in user only

Traversy · Full Stack App · ~20min

WEEK 3 —File Upload, Search, UX Polish

Days 15–21

Day 15 — Files

File Upload Full Stack

- Spring: MultipartFile endpoint, save to disk
- React: file input + FormData POST
- Display uploaded image URL from response

Amigoscode · File Upload · ~20min

Day 16 — Spring

Search & Filter Backend

- JPA query: findByTitleContainingIgnoreCase
- Accept ?search= query param in controller
- Return filtered results as JSON list

Java Brains · JPA Query · ~18min

Day 17 — React

Search & Filter Frontend

- Controlled search input with useState
- Debounce: wait 300ms before calling API
- Display filtered task list in real time

Web Dev Simplified · Debounce · ~12min

Day 18 — UX

Pagination Frontend & Backend

- Spring: Pageable — return Page<Task>
- React: page state, prev/next buttons
- Display page X of Y from API response

Java Brains · Pagination · ~18min

Day 19 — Quality

Form Validation Both Sides

- React: field-level errors with React Hook Form
- Spring: @Valid + @NotBlank annotations
- Return structured 400 errors to frontend

Web Dev Simplified · Forms · ~18min

Day 20 — UX

Error Handling & Loading States

- React: isLoading, error, data state pattern
- Show spinner during fetch, error message on fail
- Spring: @ControllerAdvice global error response

Web Dev Simplified · Async State · ~15min

Day 21 — Project

Week 3 Project: Feature Complete App

- Add search, pagination, file upload to tasks
- Proper validation errors on frontend + backend
- Clean error handling across the whole app

Traversy · Full Stack · ~20min

WEEK 4 —Testing, Docker, Deploy, CI/CD

Days 22–30

Day 22 — Testing

Testing the Backend (JUnit)

- Test Service layer with Mockito mocks
- Test Controller layer with MockMvc
- Assert HTTP status codes and response body

Amigoscode · JUnit + Spring · ~20min

Day 23 — Deploy

Dockerize the Backend

- Write Dockerfile for Spring Boot app
- docker build → docker run on port 8080
- Test all API endpoints still work in container

Amigoscode · Docker Spring · ~20min

Day 24 — Deploy

Deploy Backend to Railway

- Push Docker image to Docker Hub
- Create Railway project → link Docker image
- Add DATABASE_URL, JWT_SECRET env variables

Traversy · Railway Deploy · ~15min

Day 25 — Deploy

Deploy Frontend to Vercel

- Set REACT_APP_API_URL to Railway backend URL
- npm run build → push to GitHub
- Connect GitHub repo to Vercel → auto deploy

Traversy · Vercel Deploy · ~12min

Day 26 — Ops

Environment Variables

- React: .env file with REACT_APP_ prefix
- Spring: application.properties with ${ENV_VAR}
- Never commit secrets — add .env to .gitignore

Fireship · Env Variables · ~8min

Day 27 — Docs

API Docs with Swagger

- springdoc-openapi dependency in pom.xml
- Access /swagger-ui.html — test all endpoints
- Share Swagger URL with frontend dev / PM

JavaGuides · Swagger · ~15min

Day 28 — Perf

Performance: Caching & Indexes

- @Cacheable on frequently read endpoints
- Add @Index on DB columns used in WHERE
- Measure response time before vs after

Amigoscode · Spring Cache · ~15min

Day 29 — CI/CD

CI/CD with GitHub Actions

- Workflow: push → run mvn test → build Docker
- On pass: deploy to Railway automatically
- Status badge in README — shows build status

Amigoscode · GitHub Actions · ~20min

Day 30 — Project

Final Project: Deploy & Portfolio

- Full stack app: React + Spring Boot + PostgreSQL
- JWT auth, CRUD, search, pagination, file upload
- Live URL + GitHub repo + Swagger docs in portfolio

Traversy · Full Stack Final · ~25min

Completion goal:
Ship a live full stack app: React on Vercel + Spring Boot on Railway + PostgreSQL. Share live URL + GitHub in your portfolio.